

# PE-EC802C — VLSI Design Automation

Semester VIII • 3L:0T:0P • 3 Credits

## Module 5: High-Level Synthesis

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

---

### Contents

---

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>1</b> | <b>High-Level Synthesis (HLS) Core Concepts</b> | <b>2</b> |
| 1.1      | Core Compilation Steps                          | 2        |
| 1.2      | Hardware & Architecture Models in HLS           | 2        |
| 1.3      | Internal Representations                        | 2        |
| <b>2</b> | <b>Scheduling Algorithms</b>                    | <b>3</b> |
| 2.1      | 1. ASAP Scheduling (As-Soon-As-Possible)        | 3        |
| 2.2      | 2. ALAP Scheduling (As-Late-As-Possible)        | 3        |
| 2.3      | 3. List Scheduling                              | 4        |
| 2.4      | 4. Force-Directed Scheduling (FDS)              | 4        |
| <b>3</b> | <b>Hardware Binding &amp; Transformations</b>   | <b>4</b> |
| 3.1      | The Assignment (Binding) Problem Formulation    | 4        |
| 3.2      | 1. Register Binding (Left-Edge Algorithm)       | 5        |
| 3.3      | 2. High-Level Optimisation Transformations      | 5        |

## 1. High-Level Synthesis (HLS) Core Concepts

### High-Level Synthesis

**High-Level Synthesis (HLS)** is the automated design process that translates an algorithmic description of behavior (written in C, C++, SystemC) into a Register Transfer Level (RTL) description (written in Verilog or VHDL) implementing a digital circuit controller and datapath.

### 1.1 Core Compilation Steps

The HLS process consists of three main subproblems that are closely coupled:

1. **Scheduling:** Assigning each operation in the algorithm to a specific execution control step (clock cycle) while respecting data dependencies and resource bounds.
2. **Allocation:** Selecting the types and quantities of hardware resources (e.g. 2 adders, 1 multiplier) to implement the operations, registers, and buses.
3. **Binding (Assignment):** Mapping each operation to a specific allocated functional unit (e.g., mapping multiplication  $v_3$  to multiplier  $M_1$ ) and mapping variables to specific registers.

### 1.2 Hardware & Architecture Models in HLS

HLS tools map behavioral code onto specific structural hardware templates. The primary target model is the **Controller-Datapath Model** (Finite State Machine with Datapath, FSM-D):

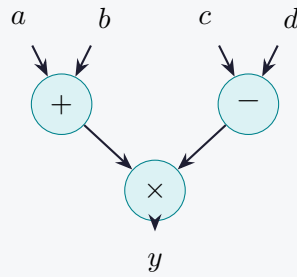
- **Datapath:** Contains functional units (ALUs, multipliers, adders) for arithmetic execution, registers/memories for data storage, and multiplexers/buses for routing data.
- **Controller:** An FSM that generates control signals (write enables, mux select lines) for each control step and reads status signals (flags, comparisons) from the datapath.

Microarchitectural models of functional units include:

- **Combinational Units:** Complete execution within a single clock cycle (e.g. basic adders).
- **Multi-cycle Units:** Take  $d \geq 2$  clock cycles to execute. No new inputs can be accepted until execution finishes (e.g. non-pipelined dividers).
- **Pipelined Units:** Take  $d \geq 2$  clock cycles but can accept new inputs every  $II$  (Initiation Interval) cycles (e.g. pipelined multipliers).

### 1.3 Internal Representations

- **Data Flow Graph (DFG):** A directed acyclic graph where vertices represent arithmetic operations ( $+$ ,  $-$ ,  $\times$ ) and directed edges represent data dependencies (values passed from one operation to another).
- **Control Data Flow Graph (CDFG):** An extension of the DFG that includes control flow structures (loops, conditional branches 'if-else', jumps). Vertices represent basic blocks of operations, and edges represent control decisions.

**Data Flow Graph for  $y = (a + b) \cdot (c - d)$** 

## 2. Scheduling Algorithms

Scheduling decides the overall trade-off between latency (clock cycles) and circuit area (number of functional units).

### 2.1 1. ASAP Scheduling (As-Soon-As-Possible)

ASAP is an unconstrained scheduling algorithm that schedules each operation to the earliest possible control step as soon as all its predecessor inputs are available.

- **Formula:** Start time  $t_i = 1$  if node  $i$  has no predecessors; otherwise:

$$t_i = \max_{j \in \text{Pred}(i)} (t_j + d_j) \quad (1)$$

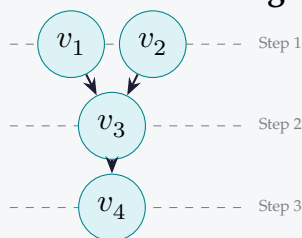
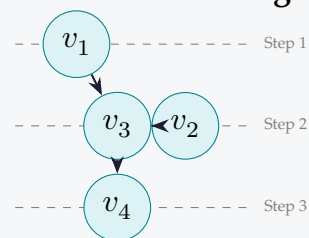
where  $d_j$  is the execution delay of predecessor node  $j$  (usually 1 cycle).

### 2.2 2. ALAP Scheduling (As-Late-As-Possible)

ALAP schedules each operation to the latest possible control step without violating a predefined maximum latency constraint  $L$  cycles.

- **Formula:** Start time  $t_i = L$  if node  $i$  has no successors; otherwise:

$$t_i = \min_{j \in \text{Succ}(i)} (t_j) - d_i \quad (2)$$

**ASAP vs. ALAP Scheduling Timelines (Latency constraint  $L = 3$ )****ASAP Scheduling:****ALAP Scheduling:**

The range between ASAP and ALAP start times ( $[t_{ASAP}, t_{ALAP}]$ ) represents the **slack** or mobility of an operation. If  $t_{ASAP} = t_{ALAP}$ , the operation lies on the critical path and has zero mobility.

### 2.3 3. List Scheduling

A resource-constrained heuristic scheduling algorithm. It resolves scheduling step-by-step using a priority queue:

1. Compute priority metrics for all operations (e.g. mobility, path length to sink).
2. For each control step  $t = 1, 2, \dots$ :
  - Identify the **ready list** of operations whose predecessors have completed execution.
  - Sort the ready list in descending order of priority.
  - Schedule the highest priority operations to the available functional units.
  - Defer the remaining ready operations to the next control step.

### 2.4 4. Force-Directed Scheduling (FDS)

FDS is a latency-constrained algorithm that minimizes resource costs by distributing operations of the same type uniformly across all control steps.

- **Probability:** The probability that operation  $i$  is scheduled in control step  $t$  is  $p_i(t) = \frac{1}{\mu_i + 1}$  for  $t \in [t_{ASAP}, t_{ALAP}]$ , where  $\mu_i$  is mobility.
- **Distribution Graph (DG):** The sum of probabilities of all operations of type  $op$  at step  $t$ :  $DG(t) = \sum_i p_i(t)$ .
- **Forces:** Calculates the attractive and repulsive forces between operations and control steps to place operations in slots with low distribution density, smoothing resource utilization.

## 3. Hardware Binding & Transformations

---

Binding maps variables to registers and operations to functional units.

### 3.1 The Assignment (Binding) Problem Formulation

The binding (assignment) problem maps operations to functional units (resource allocation constraint) and variables to registers (storage allocation constraint). It can be formulated in different mathematical ways:

- **Graph Coloring / Max-Clique Formulation:**
  - Create a **Compatibility Graph** where vertices represent operations (or variables) and edges connect vertices that can share the same resource (i.e., operations scheduled in different control steps, or variables with non-overlapping lifetimes).
  - Finding a minimum clique partition in the compatibility graph (or coloring the conflict/clash graph) yields the optimal number of resources.

- **Bipartite Matching Formulation:**

- Maps operations to compatible functional units as a bipartite matching problem to minimize total steering logic (multiplexers and buses).

- **Integer Linear Programming (ILP) Formulation:**

- Let  $x_{v,f} = 1$  if operation  $v$  is bound to functional unit  $f$ .
- Subject to:  $\sum_f x_{v,f} = 1$  (every operation is bound to exactly one unit).
- Subject to:  $x_{v_1,f} + x_{v_2,f} \leq 1$  for any two operations  $v_1, v_2$  scheduled in the same control step (resource sharing constraint).

### 3.2 1. Register Binding (Left-Edge Algorithm)

Variables that do not have overlapping lifetimes can share the same hardware register.

- **Lifetime Interval:** The span between the control step where the variable is generated and the control step where it is last read.
- Left-Edge algorithm binds overlapping lifetime intervals to a minimum set of registers, identical to channel routing track allocation.

### 3.3 2. High-Level Optimisation Transformations

Before scheduling, HLS tools optimize the algorithmic code structure to improve performance:

- **Loop Unrolling:** Duplicates loop bodies to expose parallel operations, reducing execution latency at the cost of larger circuit area.
- **Loop Pipelining:** Overlaps the execution of subsequent loop iterations in a pipelined fashion, starting iteration  $k + 1$  before iteration  $k$  finishes.
- **Tree Height Reduction:** Reorganizes expression evaluations using algebraic properties (associativity, commutativity) to balance parsing trees and reduce critical path delay.

## Quick Revision — Module 5

High-Level Synthesis | PE-EC802C

### Key Formulas and Metrics at a Glance

|                            |                                                                                 |
|----------------------------|---------------------------------------------------------------------------------|
| <b>Operation Mobility</b>  | $\mu_i = t_{i,ALAP} - t_{i,ASAP}$ (slack or scheduling freedom)                 |
| <b>ASAP Scheduling</b>     | $t_{i,ASAP} = \max_{j \in \text{Pred}(i)} (t_{j,ASAP} + d_j)$                   |
| <b>ALAP Scheduling</b>     | $t_{i,ALAP} = \min_{k \in \text{Succ}(i)} (t_{k,ALAP}) - d_i$                   |
| <b>FDS Operation Prob.</b> | $p_i(t) = \frac{1}{\mu_i + 1} \quad \text{for } t \in [t_{i,ASAP}, t_{i,ALAP}]$ |

### One-Line Definitions

- **Datapath:** The hardware block containing ALUs, multipliers, registers, and multiplexers that process data.
- **Controller:** The FSM block that generates control signals to coordinate datapath operations.
- **ASAP:** An unconstrained scheduling method that schedules operations at their earliest possible cycle.
- **ALAP:** A latency-constrained scheduling method that schedules operations at their latest possible cycle.
- **FDS:** An algorithm that minimizes functional units by distributing operations evenly across cycles.
- **Allocation:** Specifying the total hardware resources available to implement the design.
- **Binding:** Assigning logical operations and variables to specific physical functional units and registers.
- **Loop Unrolling:** A compiler optimization that replicates loop bodies to increase parallel operation opportunities.
- **Tree Height Reduction:** Reducing the depth of an expression tree using algebraic laws to speed up evaluation.

### Frequently Asked / Viva Questions

#### Q1. What are the three main subproblems in High-Level Synthesis?

(1) Scheduling (assigning operations to clock cycles), (2) Allocation (determining the type and number of hardware resources), and (3) Binding (assigning operations and variables to specific hardware units).

**Frequently Asked / Viva Questions (contd.)****Q2. Explain how register allocation can be modeled as a Graph Coloring problem.**

A variable lifetime conflict graph is constructed where vertices represent variables, and an edge connects two vertices if their lifetimes overlap. Coloring this graph such that no two adjacent vertices share the same color maps directly to register binding. The minimum number of colors corresponds to the minimum number of registers needed.

**Q3. What is mobility in High-Level Synthesis scheduling, and why is it important?**

Mobility is the difference between an operation's ALAP and ASAP start times. It represents the scheduling freedom (slack) of the operation. Zero mobility means the operation lies on the critical path, while high mobility allows the scheduler to shift the operation to resolve resource bottlenecks.

**Q4. Explain the concept of Loop Pipelining in HLS.**

Loop pipelining allows successive loop iterations to execute concurrently without waiting for the previous iteration to finish. The rate at which new iterations start is called the Initiation Interval (II). Pipelining increases throughput dramatically.

**Q5. What is the difference between Resource-Constrained and Latency-Constrained scheduling?**

Resource-constrained scheduling minimizes latency given a fixed limit on hardware units (e.g. List Scheduling). Latency-constrained scheduling minimizes the hardware unit cost given a fixed maximum execution time (e.g. ALAP, FDS).

**Q6. How does Tree Height Reduction reduce critical path delay?**

Consider evaluating  $a + b + c + d$ . Sequential evaluation  $((a + b) + c) + d$  has a depth of 3 adder stages. Using associativity, we can compute  $(a + b) + (c + d)$  in a tree format with a depth of only 2 stages, reducing execution delay.

**Q7. What is functional unit binding, and what is its goal?**

Functional unit binding assigns scheduled operations (e.g. additions) to specific physical components (e.g., Adder 1). The goal is to maximize sharing, which minimizes the total area of functional units, multiplexers, and interconnect buses.

**Q8. Explain how the Left-Edge algorithm is used in Register Binding.**

It sorts variables by the start of their lifetimes. It then assigns non-overlapping variable lifetimes to the same register track sequentially, maximizing register sharing and minimizing the total register count.

**Q9. What is the role of the FSM controller in the generated RTL netlist?**

The Finite State Machine (FSM) controller generates the control signals (e.g. multiplexer select lines, register write enables) at each clock cycle, coordinating the datapath to execute operations in the scheduled order.

**Q10. Why is high-level synthesis preferred over writing manual RTL?**

**Frequently Asked / Viva Questions (contd.)**

HLS allows designing at a higher level of abstraction (C/C++), which speeds up development, makes verification easier, and allows designers to quickly explore different trade-offs between area and speed by changing scheduling constraints.

**Comparison Snapshot****ASAP vs. ALAP Scheduling:**

| Feature             | ASAP Scheduling                                | ALAP Scheduling                                      |
|---------------------|------------------------------------------------|------------------------------------------------------|
| <b>Constraints</b>  | Unconstrained (assumes infinite resources).    | Latency-constrained (limited to $L$ cycles).         |
| <b>Goal</b>         | Finds the minimum possible latency boundary.   | Finds the latest start times to delay operations.    |
| <b>Search Order</b> | Topological search starting from source nodes. | Reverse topological search starting from sink nodes. |

**DFG vs. CDFG:**

| Feature               | Data Flow Graph (DFG)                                    | Control Data Flow Graph (CDFG)                         |
|-----------------------|----------------------------------------------------------|--------------------------------------------------------|
| <b>Information</b>    | Models only data dependencies and arithmetic operations. | Models both data dependencies and control decisions.   |
| <b>Branches/Loops</b> | Cannot represent loops or conditional branches.          | Explicitly models loop jumps and conditional branches. |
| <b>Use Case</b>       | Used for basic scheduling and binding.                   | Used for full behavioral synthesis of systems.         |